ARMIS Platform

Developer README — Project Overview & Local Setup

1. What Is ARMIS?

ARMIS (Advanced Resource & Mission Intelligence System) is a microservices-based backend platform developed by DefenceTech A.S. It provides real-time situational awareness, asset tracking, and mission planning capabilities for Turkish defense customers.

The platform exposes a REST API consumed by the ARMIS Web Console (React) and ARMIS Mobile (React Native). All inter-service communication uses gRPC internally; external clients use HTTPS REST endpoints via the API Gateway.

2. High-Level Architecture

ARMIS consists of six backend microservices:

Service	Port	Responsibility
api-gateway	8080	TLS termination, JWT validation, rate limiting, routing
auth-service	9001	User authentication, role management, token issuance
asset-service	9002	Asset registry, real-time location tracking via WebSocket
mission-service	9003	Mission CRUD, assignment, status lifecycle
notification-service	9004	Push/email alerts triggered by mission events
report-service	9005	Async PDF/Excel report generation, S3 upload

3. Technology Stack

Layer	Technology
Language	Java 17
Framework	Spring Boot 3.2, Spring Security, Spring Data JPA
Database	PostgreSQL 15 (per-service schema isolation)
Cache	Redis 7 (session tokens, rate limit counters)
Message Broker	RabbitMQ 3.12 (event-driven notifications)
Container	Docker + Docker Compose (local), Kubernetes (production)
Service Mesh	Istio (mTLS between services in production)
CI/CD	GitLab CI — .gitlab-ci.yml in project root

4. Prerequisites

Before cloning the repository, ensure the following tools are installed:

- Java 17 (OpenJDK recommended) — java -version should show 17.x
- Maven 3.9+ — mvn -version
- Docker Desktop 4.x and Docker Compose v2
- Git 2.40+
- IntelliJ IDEA (recommended) or VS Code with Java Extension Pack

5. Local Setup

5.1 Clone the Repository

```
git clone https://gitlab.defencetech.com.tr/armis/armis-platform.git
cd armis-platform
```

5.2 Start Infrastructure Services

The docker-compose.infra.yml file starts PostgreSQL, Redis, and RabbitMQ locally. You do not run the application services inside Docker during development — they run directly on your machine via Maven.

```
docker compose -f docker-compose.infra.yml up -d
```

Verify all three containers are healthy:

```
docker compose -f docker-compose.infra.yml ps
```

5.3 Configure Environment Variables

Copy the sample env file and fill in secrets. Never commit .env to Git.

```
cp .env.sample .env
```

Required variables in .env:

- DB_PASSWORD — PostgreSQL password (default for local: armis_dev)
- JWT_SECRET — minimum 256-bit random string
- RABBITMQ_DEFAULT_PASS — RabbitMQ password (default: guest for local)
- S3_BUCKET / S3_REGION / AWS_ACCESS_KEY_ID — only needed for report-service

5.4 Run Database Migrations

Each service manages its own schema with Flyway. Run migrations for all services:

```
./scripts/migrate-all.sh
```

To migrate a single service only:

```
./scripts/migrate.sh auth-service
```

5.5 Start Services

Each service is a separate Maven module. To start all services simultaneously using the helper script:

```
./scripts/start-all.sh
```

To start a specific service in your IDE, run the main class in the service module, e.g.:

```
com.defencetech.armis.auth.AuthServiceApplication
```

5.6 Verify the Setup

Once all services are running, test the health endpoints:

```
curl http://localhost:8080/actuator/health
```

Expected response: {"status":"UP"}

The API Gateway health check aggregates all downstream service statuses. If any service is down, the gateway will report DEGRADED.

6. Common First-Day Issues

Problem	Solution
Port 8080 already in use	Run: lsof -i :8080 and kill the conflicting process
Flyway migration fails	Check DB_PASSWORD in .env matches docker-compose.infra.yml
JWT validation errors	JWT_SECRET must be identical across all services
RabbitMQ connection refused	Ensure infra containers are running: docker compose ps
Services can't reach each other	Use localhost; inter-service DNS only works in Kubernetes

7. Project Structure

```
armis-platform/
  api-gateway/          # Spring Cloud Gateway
  auth-service/         # Spring Boot + Spring Security
  asset-service/        # Spring Boot + WebSocket
  mission-service/      # Spring Boot
  notification-service/ # Spring Boot + RabbitMQ consumer
  report-service/       # Spring Boot + async tasks
  shared/               # Shared DTOs, exceptions, utils
  scripts/              # Dev helper scripts
  docker-compose.infra.yml
  .gitlab-ci.yml
```

8. Useful Contacts

- Platform Lead: Emre Yildirim (emre.yildirim@defencetech.com.tr)
- DevOps / Infra: Canan Arslan (canan.arslan@defencetech.com.tr)
- Slack: #armis-dev channel for day-to-day questions
- Confluence: https://confluence.defencetech.com.tr/armis (internal network only)